

UNIVERSIDAD PERUANA LOS ANDEZ

INFOGRAFIA DE CAPITULOS 5-9

ALUMNO: VILCAPOMA OCHOA MICHAEL AXEL

Cómo conviven e interactúan distintas plataformas y lenguajes con el protocolo seguro de SQL Azure.



Cuadro Comparativo de Tecnologías de Conexión

Tecnología	Propósito Principal	Característica Destacada en SQL Azure
SQLCMD	Administración y Scripts	Ejecución rápida por consola utilizando el formato usuario@servidor.
ADO.NET	Aplicaciones .NET nativas	Control total y máximo rendimiento mediante controladores optimizados.
Entity Framework	Desarrollo Orientado a Objetos	Abstracción de datos con LINQ, reduciendo la necesidad de escribir T-SQL manual.
PHP	Aplicaciones Web Dinámicas	Soporte nativo a través del driver Microsoft Drivers for PHP for SQL Server.
JDBC	Aplicaciones Java	Utilización de la biblioteca oficial para conexiones robustas e interplataforma.



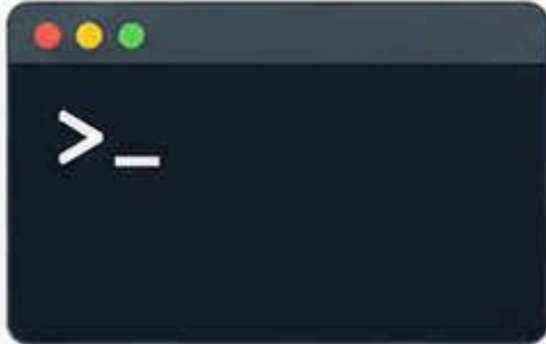
Idea clave

Sin importar la tecnología utilizada, SQL Azure exige comunicación mediante **TDS 7.3+** y cifrado **SSL/TLS** para garantizar interoperabilidad, seguridad y acceso confiable.



Conexión por línea de comandos para automatización, despliegues ágiles y tareas administrativas en SQL Azure.

1 Sintaxis de Conexión Directa



```
# Conexión inicial a la base de datos master de tu servidor lógico
sqlcmd -S sistemagrados.database.windows.net -U administrador@sistemagrados \
-P "ClaveSegura123!" -d master
```

-S Servidor lógico

-U usuario@servidor

-P contraseña

-d master

Bash

2 Flujo de Administración por Consola



Administrador / Desarrollador



SQLCMD



Conexión segura
SSL/TLS



Servidor lógico
de Azure SQL



Base de datos
master



scripts



consultas



automatización



despliegue



SQLCMD es ideal para administración rápida, scripts repetibles y automatización.

3 Restricciones Críticas de SQLCMD en la Nube

1 Sin Autenticación Windows

SQL Azure no admite cuentas de dominio locales ni autenticación integrada clásica a través de SQLCMD; es obligatorio el uso de credenciales SQL Server o tokens de Microsoft Entra ID.



Autenticación
Windows: NO
soportada



Autenticación SQL
o Microsoft Entra ID:
soportada

2 Limitación de Parámetros

Los parámetros -z y -Z no están soportados en SQL Azure para cambio físico de contraseñas.



SQL

```
ALTER LOGIN administrador WITH PASSWORD = 'NuevaClaveSegura123!';
```



El cambio de contraseña debe ejecutarse mediante código T-SQL.



Idea clave

SQLCMD ofrece administración rápida y automatizable en SQL Azure, pero exige autenticación compatible con la nube y respeta restricciones específicas del servicio.



1

ARQUITECTURA DE CONEXIÓN
MULTI-TECNOLOGÍA A SQL AZURE

Diferentes plataformas y lenguajes de programación se conectan a SQL Azure de forma segura y estandarizada.

PROTOCOLO COMÚN DE COMUNICACIÓN



CUADRO COMPARATIVO DE TECNOLOGÍAS DE CONEXIÓN

TECNOLOGÍA	PROPÓSITO PRINCIPAL	CARACTERÍSTICA DESTACADA EN SQL AZURE
SQLCMD	Administración y Scripts	Ejecución rápida por consola utilizando el formato usuario@servidor.
ADO.NET	Aplicaciones .NET nativas	Control total y máximo rendimiento mediante controladores optimizados.
Entity Framework	Desarrollo Orientado a Objetos	Abstracción de datos con LINQ, reduciendo la necesidad de escribir T-SQL manual.
PHP	Aplicaciones Web Dinámicas	Soporte nativo a través del driver Microsoft Drivers for PHP for SQL Server.
JDBC	Aplicaciones Java	Utilización de la biblioteca oficial para conexiones robustas e interplataforma.

Seguridad: SSL/TLS Obligatorio

Protocolo: TDS 7.3+

Puerto: TCP 1433

2

EL ENFOQUE DE CONSOLA:
ADMINISTRACIÓN RÁPIDA CON SQLCMD

Ideal para tareas de automatización, scripts y despliegues rápidos sin necesidad de interfaz gráfica.

SINTAXIS DE CONEXIÓN DIRECTA

Para interactuar mediante consola, especifica el servidor remoto, las credenciales en formato @servidor y apunta inicialmente a la base de datos master.

```
# Conexión inicial a la base de datos master
# de tu servidor lógico

sqlcmd -S sistemagrados.database.windows.net \
-U administrador@sistemagrados \
-P "ClaveSegura123!" \
-d master
```



RESTRICCIONES CRÍTICAS DE SQLCMD EN LA NUBE

1 SIN AUTENTICACIÓN WINDOWS



SQL Azure no admite cuentas de dominio locales o autenticación integrada clásica a través de SQLCMD; es obligatorio el uso de credenciales SQL Server o tokens de Microsoft Entra ID.

2 LIMITACIÓN DE PARÁMETROS

Los parámetros -z y -Z (usados en entornos locales para el cambio físico de contraseñas) no están soportados.

Los cambios de contraseña se ejecutan mediante código T-SQL:

```
ALTER LOGIN administrador
WITH PASSWORD = 'NuevaClaveSegura123!';
```



3

CONEXIÓN APLICADA EN C# CON ADO.NET
(SISTEMA DE GRADOS Y TÍTULOS)

Implementación del código de conexión en .NET con control de excepciones y buenas prácticas.

1 CONSTRUCCIÓN SEGURA DE LA CADENA DE CONEXIÓN

C#

```
using Microsoft.Data.SqlClient;

// Construcción dinámica de la cadena de conexión
SqlConnectionStringBuilder builder = new SqlConnectionStringBuilder();
builder.DataSource = "sistemagrados.database.windows.net,1433";
builder.InitialCatalog = "SistemaGradosTitulos";
builder.UserID = "usuario_desarrollo";
builder.Password = "ContraseñaSegura99*";
builder.Encrypt = true; // Cifrado SSL obligatorio
builder.TrustServerCertificate = false; // Validación estricta del certificado del servidor

string connectionString = builder.ConnectionString;
```

2 PRUEBA DE CONEXIÓN Y CONTROL DE EXCEPCIONES RESILIENTE

C#

```
try
{
    // El bloque using asegura el cierre del socket al terminar el proceso
    using (SqlConnection conexion = new SqlConnection(connectionString))
    {
        Console.WriteLine("Conectando con el servidor en la nube...");
        conexion.Open();

        Console.WriteLine("-----");
        Console.WriteLine($"Conexión Exitosa con: " + conexion.Database + "!");
        Console.WriteLine($"Estado de la Conexión: " + conexion.State);
        Console.WriteLine("-----");
    }
}
catch (SqlException ex)
{
    // Captura de errores específicos de SQL Azure (Firewall, login, timeout)
    Console.WriteLine("\n[ERROR DE CONEXIÓN]");
    Console.WriteLine("Código de Error: " + ex.Number);
    Console.WriteLine("Mensaje: " + ex.Message);
}
catch (Exception ex)
{
    // Captura de otros errores de infraestructura o ejecución
    Console.WriteLine("\n[ERROR GENERAL]: " + ex.Message);
}
```

BUENAS PRÁCTICAS CLAVE PARA EL DESARROLLADOR EN LA NUBE

CONNECTION POOLING (AGRUPACIÓN)
ADO.NET lo maneja de manera automática. No lo deshabilites; ayuda a reutilizar conexiones activas y disminuye la latencia de negociación TCP.

CIERRE EXPLÍCITO
Utiliza siempre bloques using en C#. Una conexión inactiva en la nube se cerrará automáticamente a los 30 minutos, pero liberar el recurso inmediatamente en el cliente evita saturar los límites de tu plan de Azure SQL.

CREACIÓN DE LA TABLA SOLICITUD

```
-- Creación de la tabla principal del módulo de trámites
CREATE TABLE Tramite.Solicitud
(
  IdSolicitud      INT IDENTITY(1,1) PRIMARY KEY,
  IdEstudiante     INT NOT NULL,
  IdTipoTramite    TINYINT NOT NULL,
  IdEstadoSolicitud TINYINT NOT NULL,
  FechaRegistro   DATETIME2 NOT NULL
  CONSTRAINT DF_Solicitud_Fecha DEFAULT SYSDATETIME(),
  NumeroExpediente VARCHAR(20) NOT NULL,
  Observaciones   NVARCHAR(500) NULL,
  MontoPago       DECIMAL(10,2) NULL
);
GO
```

ESQUEMA DE LA TABLA			
Columna	Tipo de Dato	Nulo	Descripción
IdSolicitud	INT IDENTITY(1,1)	NO	Identificador único de la solicitud (Clave primaria)
IdEstudiante	INT	NO	Clave foránea hacia el estudiante
IdTipoTramite	TINYINT	NO	Tipo de trámite solicitado
IdEstadoSolicitud	TINYINT	NO	Estado actual de la solicitud
FechaRegistro	DATETIME2	NO	Fecha y hora de registro
NumeroExpediente	VARCHAR(20)	NO	Código del expediente
Observaciones	NVARCHAR(500)	SÍ	Observaciones adicionales
MontoPago	DECIMAL(10,2)	SÍ	Monto pagado por el trámite

Todos los tipos de datos utilizados están optimizados para SQL Azure, garantizando rendimiento, compatibilidad y eficiencia en costos.

JUSTIFICACIÓN ARQUITECTÓNICA DE LOS TIPOS DE DATOS			
TIPO DE DATO	USADO EN	CARACTERÍSTICAS PRINCIPALES	JUSTIFICACIÓN EN EL SISTEMA
INT (Identificadores)	- IdSolicitud - IdEstudiante	- Rango: -2,147,483,648 a 2,147,483,647 - Almacenamiento: 4 bytes - Ideal para llaves primarias y foráneas	✓ Proporciona un rango sumamente amplio (más de 2,000 millones de registros posibles) con un consumo de apenas 4 bytes por fila, ideal para llaves primarias con auto-incremento.
TINYINT (Catálogos Pequeños)	- IdTipoTramite - IdEstadoSolicitud	- Rango: 0 a 255 - Almacenamiento: 1 byte - Muy eficiente para catálogos con pocas opciones	✓ Almacena valores enteros de 0 a 255 consumiendo solo 1 byte. Es la opción más eficiente para catálogos con pocas opciones (como los estados: Iniciado, Pendiente, Aprobado, Rechazado).
DATETIME2 (Fechas)	FechaRegistro	- Precisión: Hasta 7 decimales de segundo - Rango: 0001-01-01 a 9999-12-31 - Almacenamiento: 6 a 8 bytes (variable)	✓ Reemplaza al antiguo DATETIME. Tiene una precisión configurable de hasta 7 decimales de segundo y un rango de fechas más amplio (desde el año 0001), con un almacenamiento variable y altamente eficiente.
VARCHAR(n) (Códigos de Estructura Fija)	NumeroExpediente	- Almacena cadenas alfanuméricas no Unicode - Almacenamiento: 1 byte por carácter + 2 bytes de longitud	✓ Se utiliza para cadenas de caracteres estándar (no Unicode) donde no se requieren caracteres especiales o tildes, ahorrando 1 byte por carácter en comparación con su contraparte NVARCHAR.
NVARCHAR(n) (Texto Multilingüe)	Observaciones	- Almacena cadenas Unicode - Almacenamiento: 2 bytes por carácter + 2 bytes de longitud	✓ Utiliza codificación Unicode (2 bytes por carácter). Es indispensable para campos de texto libre donde los usuarios puedan ingresar tildes, la letra "ñ" o caracteres especiales de otros idiomas sin que se corrompa el texto.
DECIMAL(p,s) (Montos Financieros)	MontoPago	- Exacto y de precisión fija - p = dígitos totales (10) - s = dígitos decimales (2) - Almacenamiento: Variable (5 a 9 bytes)	✓ Almacena números de precisión numérica exacta con 10 dígitos en total y 2 decimales. Evita los errores de redondeo físico y pérdida de centavos que sufren los tipos de datos de punto flotante como FLOAT o REAL.

CONSULTA DE CONTROL DE TRÁMITES POR ESTADO

Para monitorear el flujo de expedientes y saber cuáles son los estados con mayor carga de trabajo, aplicamos funciones de agregado combinadas con un acoplamiento de tablas (INNER JOIN).

```
SELECT
es.NombreEstado,
COUNT(*) AS CantidadSolicitudes,
MIN(s.FechaRegistro) AS PrimeraSolicitud,
MAX(s.FechaRegistro) AS UltimaSolicitud
FROM Tramite.Solicitud s
INNER JOIN Catalogo.EstadoSolicitud es
ON es.IdEstadoSolicitud = s.IdEstadoSolicitud
GROUP BY es.NombreEstado;
```

SQL

Esta consulta nos permite obtener un resumen ejecutivo del estado actual del sistema de trámites con una sola instrucción T-SQL.

¿CÓMO FUNCIONA LA CONSULTA?

1 FUENTE DE DATOS

Tramite.Solicitud (s)

- Registros de solicitudes

Catalogo.EstadoSolicitud (es)

- Catálogo de estados del trámite

2 INNER JOIN

Se acoplan las tablas por la clave foránea IdEstadoSolicitud.

3 AGRUPACIÓN

Los registros se agrupan por NombreEstado.

4 AGREGACIÓN

Se aplican las funciones COUNT, MIN y MAX para obtener métricas por grupo.

RESULTADO EJEMPLO

NombreEstado	CantidadSolicitudes	PrimeraSolicitud	UltimaSolicitud
Iniciado	126	2024-12-01 08:15:23.1200000	2025-05-21 14:35:10.4500000
Pendiente	245	2024-11-20 09:02:11.3300000	2025-05-22 10:11:05.2200000
En Revisión	98	2024-12-05 16:45:12.7800000	2025-05-22 11:42:30.6600000
Aprobado	312	2024-09-15 10:05:01.0000000	2025-05-20 17:55:45.9100000
Rechazado	37	2024-10-10 13:20:33.5500000	2025-05-18 09:33:22.0100000

Esta información ayuda a la toma de decisiones y a balancear la carga de trabajo del personal administrativo.

FUNCIONES ESCALARES Y DE AGREGADO UTILIZADAS

1 COUNT(*)

Función de Agregado

Cuenta el número total de filas que pertenecen a cada grupo de estado.

- Cuenta todas las filas, incluyendo las que puedan tener valores NULL en otras columnas.

2 MIN()

Función de Agregado

Evalúa el conjunto de fechas DATETIME2 para extraer el registro más antiguo (el primero en ingresar).

- Devuelve la fecha y hora más pequeña del grupo.

3 MAX()

Función de Agregado

Evalúa el conjunto de fechas DATETIME2 para extraer el registro más reciente (el último registrado).

- Devuelve la fecha y hora más grande del grupo.

BENEFICIOS DE USAR FUNCIONES DE AGREGACIÓN

- Reducción de la cantidad de datos transferidos.
- Consultas más rápidas y eficientes.
- Aprovecha los índices existentes para procesamiento de grandes volúmenes.
- Ideal para dashboards y reportes ejecutivos en tiempo real.

AUDITORÍA ESTRUCTURADA DEL DICCIONARIO DE DATOS

En SQL Azure, el acceso a las tablas físicas del sistema está restringido para proteger la infraestructura multi-inquilino (multi-tenant). Por ello, se recurre a las vistas estándar de SQL conocidas como **INFORMATION_SCHEMA**.

SQL

```
-- Obtener la lista completa de tablas, columnas
-- y tipos de datos del sistema
SELECT
  TABLE_SCHEMA,
  TABLE_NAME,
  COLUMN_NAME,
  DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA IN
(
  'Academico',
  'Tramite',
  'Evaluacion',
  'Documento'
)
ORDER BY TABLE_SCHEMA, TABLE_NAME, ORDINAL_POSITION;
```

BENEFICIOS DE USAR VISTAS DE CATÁLOGO EN SQL AZURE

- PORTABILIDAD**
Al apegarse al estándar ISO/ANSI SQL, estas consultas de metadatos funcionarán sin cambios si el sistema se conecta a otros motores de bases de datos compatibles.
- SEGURIDAD FÍSICA**
Abstraen los detalles de almacenamiento interno que administra Microsoft, exponiendo únicamente la configuración lógica autorizada para el usuario.
- AUDITORÍA DE ESQUEMAS**
Permite verificar dinámicamente si todas las tablas de nuestro sistema (por ejemplo, el módulo de Evaluacion o Documento) cumplen con los tipos de datos estipulados en el diseño de la arquitectura.

RESULTADO EJEMPLO: ESTRUCTURA DEL SISTEMA DE TRÁMITES

TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	DATA_TYPE	IS_NULLABLE	ORDINAL_POSITION
Academico	Estudiante	IdEstudiante	int	NO	1
		CodigoEstudiante	varchar	NO	2
		Nombres	nvarchar	NO	3
		Apellidos	nvarchar	NO	4
		FechaNacimiento	date	YES	5
Tramite	Solicitud	IdSolicitud	int	NO	1
		IdEstudiante	int	NO	2
		IdTipoTramite	tinyint	NO	3
		IdEstadoSolicitud	tinyint	NO	4
		FechaRegistro	datetime2	NO	5
		NumeroExpediente	varchar	NO	6
		Observaciones	nvarchar	YES	7
		MontoPago	decimal	YES	8
Evaluacion	EvaluacionAcademica	IdEvaluacion	int	NO	1
		IdSolicitud	int	NO	2
		Promedio	decimal	YES	3
		FechaEvaluacion	datetime2	NO	4
Documento	ArchivoDocumento	IdDocumento	int	NO	1
		IdSolicitud	int	NO	2
		NombreArchivo	nvarchar	NO	3
		RutaArchivo	nvarchar	NO	4
		FechaCarga	datetime2	NO	5

★ MEJORES PRÁCTICAS

- Consulta periódicamente el diccionario de datos para conocer la estructura real.
- Úsalo para generar documentación técnica automática del sistema.
- Verifica columnas, tipos de datos y nulabilidad antes de desarrollar reportes o integraciones.
- Recuerda: INFORMATION_SCHEMA muestra la lógica, no los detalles de almacenamiento físico.

EL CAMBIO DE PARADIGMA: PAAS (PLATAFORMA COMO SERVICIO)

En SQL Azure, tú te enfocas en tus datos y en tu código. Microsoft se encarga del hardware, parches de seguridad y alta disponibilidad.

TÚ CONTROLAS
(TÚ TE ENFOCAS AQUÍ)

TUS DATOS

TU CÓDIGO T-SQL Y APLICACIONES

- Esquema de la base de datos
- Consultas y procedimientos
- Funciones, vistas, triggers
- Índices y restricciones

SQL AZURE

Conexión segura SSL/TLS

MICROSOFT GESTIONA
(TÚ NO TE PREOCUPAS AQUÍ)

HARDWARE E INFRAESTRUCTURA

PARCHES Y SEGURIDAD

ALTA DISPONIBILIDAD Y BACKUP

MONITOREO Y RENDIMIENTO

Por ende, cualquier instrucción que intente interactuar con el entorno físico o el sistema operativo queda bloqueada.

COMPATIBILIDAD DE INSTRUCCIONES T-SQL		
CATEGORÍA DE INSTRUCCIÓN	ADMITIDO EN SQL AZURE	ALTERNATIVA / EXPLICACIÓN
MANIPULACIÓN DE DATOS (SELECT, INSERT, UPDATE, DELETE)	SÍ (100%)	Estándar ANSI SQL completamente operativo.
ADMINISTRACIÓN DEL SERVIDOR (ALTER SYSTEM, SHUTDOWN)	NO	Microsoft gestiona el aprovisionamiento y apagado de nodos.
ACCESO A SISTEMA OPERATIVO (xp_cmdshell)	NO	Seguridad estricta; no se permite ejecutar scripts a nivel de sistema.
PROGRAMACIÓN DE TAREAS (SQL SERVER AGENT)	NO	Reemplazado por Azure Elastic Jobs o Azure Logic Apps.
AUDITORÍA DE CONSULTAS (SQL SERVER PROFILER)	NO	Se utilizan Eventos Extendidos (XEvents) o Query Store.

¿POR QUÉ EXISTEN ESTAS RESTRICCIONES?

SQL Azure es un servicio PaaS multi-inquilino (multi-tenant). Para garantizar la seguridad, estabilidad y el rendimiento de todos los clientes, se restringe el acceso a las capas inferiores de la plataforma.

ALTERNATIVAS MODERNAS EN AZURE

Azure Elastic Jobs
Para programar tareas recurrentes y automatizaciones.

Azure Logic Apps
Para orquestar flujos de trabajo y procesos empresariales.

Azure Monitor / XEvents
Para monitoreo avanzado, auditoría y diagnóstico de consultas.

Azure Policy
Para gobernanza, cumplimiento y control de políticas.

1. PROCEDIMIENTOS DE FIREWALL EXCLUSIVOS DE SQL AZURE

Al no tener acceso a la infraestructura física de red, el firewall se administra directamente desde Transact-SQL en la base de datos **master** mediante procedimientos del sistema dedicados.

Aplicación Cliente

Firewall de SQL Azure (Administrado por T-SQL)

SQL Azure (Base de Datos)

A. AUTORIZAR IP O RANGO DE IPs

```
-- Autorizar una IP o rango de IPs para conectarse
-- a la base de datos
EXEC sp_set_firewall_rule
  @name = N'AccesoOficinaCentral',
  @start_ip_address = '192.168.1.50',
  @end_ip_address = '192.168.1.100';
```

Parámetros

- @name: Nombre descriptivo de la regla.
- @start_ip_address: IP inicial del rango permitido.
- @end_ip_address: IP final del rango permitido.

B. ELIMINAR REGLA DE FIREWALL EXISTENTE

```
-- Eliminar una regla de firewall existente
EXEC sp_delete_firewall_rule
  @name = N'AccesoOficinaCentral';
```

Nota Importante

Todos los cambios se aplican de inmediato y se gestionan a nivel de servidor lógico.

Estas instrucciones solo se ejecutan en la base de datos **master**.
Permite controlar de forma segura qué equipos pueden acceder a tu base de datos en la nube.

2. PROCEDIMIENTOS DE CATÁLOGO ESTÁNDAR (ODBC)

Para aplicaciones externas que necesitan inspeccionar el esquema de trámites sin consultar directamente tablas internas del sistema, SQL Azure provee procedimientos de catálogo **seguros y portables**.

Procedimiento	Propósito	Descripción	Ejemplo de Uso
sp_tables	Lista de tablas	Retorna la lista de todas las tablas (y vistas) en la base de datos actual. Útil para descubrir los objetos disponibles.	<pre>EXEC sp_tables;</pre>
sp_columns	Descripción de columnas	Devuelve información detallada de las columnas de una tabla específica, incluyendo tipo de dato, tamaño y nulabilidad.	<pre>EXEC sp_columns @table_name = 'Solicitud', @table_owner = 'Tramite';</pre>
sp_pkeys	Llaves primarias (PK)	Retorna las llaves primarias definidas en las tablas. Esencial para entender la estructura principal del sistema.	<pre>EXEC sp_pkeys @table_name = 'Solicitud', @table_owner = 'Tramite';</pre>
sp_fkeys	Llaves foráneas (FK)	Devuelve las llaves foráneas y las relaciones entre tablas. Permite analizar la integridad y las dependencias lógicas.	<pre>EXEC sp_fkeys @pktable_name = 'Solicitud', @pktable_owner = 'Tramite';</pre>

Beneficios de usar procedimientos de catálogo en SQL Azure

Seguridad: No exponen detalles internos del almacenamiento físico.

Portabilidad: Compatible con estándares ODBC para múltiples plataformas.

Auditoría de Esquemas: Facilitan la validación y documentemiento del sistema de trámites.



Código robusto en la nube para garantizar la integridad de los trámites académicos



1 PROCEDIMIENTO ALMACENADO CON CONTROL DE ERRORES (THROW)



Este procedimiento valida la existencia de un expediente antes de consultar la información completa. Si el expediente no existe, lanza una excepción personalizada.

```
01 CREATE OR ALTER PROCEDURE Tramite.SP_ConsultarSolicitud
02     @NumeroExpediente VARCHAR(20)
03 AS
04 BEGIN
05     -- Optimiza la transferencia de datos de red
06     -- evitando enviar mensajes de filas afectadas
07     SET NOCOUNT ON;
08
09     -- 1. Validación de Existencia
10     IF NOT EXISTS
11     (
12         SELECT 1
13         FROM Tramite.Solicitud
14         WHERE NumeroExpediente = @NumeroExpediente
15     )
16     BEGIN
17         -- Código de error personalizado (rango 50000 a 2147483647), mensaje y estado
18         THROW 50001, 'El expediente solicitado no existe.', 1;
19     END;
20
21     -- 2. Consulta de Datos Cruzados (Estudiante, Persona, Estado)
22     SELECT
23         s.NumeroExpediente,
24         p.Nombres,
25         p.ApellidoPaterno,
26         p.ApellidoMaterno,
27         et.NombreEstado,
28         s.FechaRegistro,
29         s.Observaciones
30     FROM Tramite.Solicitud s
31     INNER JOIN Academico.Estudiante e ON e.IdEstudiante = s.IdEstudiante
32     INNER JOIN Academico.Persona p ON p.IdPersona = e.IdPersona
33     INNER JOIN Catalogo.EstadoSolicitud et ON et.IdEstadoSolicitud = s.IdEstadoSolicitud
34     WHERE s.NumeroExpediente = @NumeroExpediente;
35 END;
36 GO
```

SQL

DESGLOSE DEL PROCEDIMIENTO

1 VALIDACIÓN



Verifica si el expediente existe para evitar procesos innecesarios y devolver un error claro al usuario.

2 THROW



THROW lanza una excepción personalizada con un código definido (50001) y un mensaje específico.

3 CONSULTA CRUZADA



Obtiene la información completa uniendo tablas relacionadas para presentar al usuario los datos consolidados.

¿POR QUÉ USAR THROW EN LUGAR DE RAISERROR?



Mejor compatibilidad futura
THROW es la forma moderna de generar excepciones en T-SQL.



Preserva el estado original
Mantiene intacta la información de la excepción.



Mensajes más claros
Permite enviar mensajes precisos y controlados.



2 CAPTURA DE EXCEPCIONES EN LA APLICACIÓN CLIENTE (C# / .NET)



En aplicaciones de escritorio o web, los errores de la base de datos (como el código 50001 lanzado por el procedimiento) o las pérdidas de conexión temporales deben ser controladas de manera limpia para no afectar al usuario final.

```
01 using Microsoft.Data.SqlClient;
02
03 try
04 {
05     // Código para abrir conexión y ejecutar el procedimiento almacenado
06     using (SqlCommand cmd = new SqlCommand("Tramite.SP_ConsultarSolicitud", conexion))
07     {
08         cmd.CommandType = CommandType.StoredProcedure;
09         cmd.Parameters.AddWithValue("@NumeroExpediente", txtExpediente.Text);
10
11         // Ejecución de la consulta...
12     }
13 }
14 catch (SqlException ex)
15 {
16     MessageBox.Show(ex.Message, "Error en la consulta");
17 }
```

C#

¿QUÉ ERRORES CAPTURA ESTE BLOQUE?



Errores de Lógica (THROW)

Captura excepciones lanzadas por el procedimiento almacenado, como el código 50001 cuando el expediente no existe.



Errores de Conexión

Captura caídas de red, timeouts, cambios de firewall o cualquier problema físico que impida la comunicación con SQL Azure.



3 RECOMENDACIÓN PARA LA NUBE: RETRY LOGIC (RECONEXIÓN AUTOMÁTICA)

Debido a la naturaleza de SQL Azure ejecutándose en Internet, es indispensable configurar políticas de reintento (Retry Logic) para gestionar de forma transparente las micro-caídas del servicio o los parpadeos de red.



Falla temporal de conexión



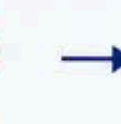
Se detecta el error



Espera con backoff exponencial



Reintenta la conexión



Conexión restablecida



BUENAS PRÁCTICAS CLAVE



Validar siempre los datos de entrada.



Usar THROW para manejar errores de negocio.



Capturar SqlException específicamente.



Implementar Retry Logic en aplicaciones cliente.



Registrar errores (logging) para auditoría.



Crea y evoluciona las estructuras de tu base de datos de forma controlada y segura en SQL Azure

1 CREACIÓN DE LA TABLA DE HISTORIAL (CREATE TABLE)

Para mantener un rastro inmutable de los cambios de estado de las solicitudes, creamos la tabla `HistorialEstado`, relacionada mediante clave foránea (FOREIGN KEY) con la tabla principal de solicitudes.

```
CREATE TABLE Tramite.HistorialEstado
(
  IdHistorial INT IDENTITY(1,1) PRIMARY KEY,
  IdSolicitud INT NOT NULL,
  IdEstadoAnterior TINYINT NULL,
  IdEstadoNuevo TINYINT NOT NULL,
  FechaCambio DATETIME2 NOT NULL
  CONSTRAINT DF_Historial_Fecha DEFAULT SYSDATETIME(),

  CONSTRAINT FK_Historial_Solicitud
  FOREIGN KEY (IdSolicitud)
  REFERENCES Tramite.Solicitud(IdSolicitud)
);
GO
```

SQL

¿QUÉ LOGRAMOS?



Auditabilidad completa:
Cada cambio de estado queda registrado.

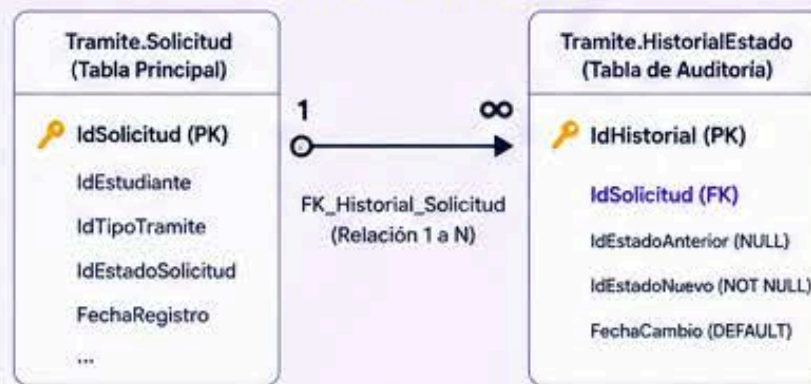


Integridad referencial:
No se puede registrar historial de una solicitud que no exista.



Inmutabilidad lógica:
El historial no se modifica, solo se inserta.

DIAGRAMA RELACIONAL



REGISTRO DE EJEMPLO

IdHistorial	IdSolicitud	Estado Ant.	Estado Nuevo	FechaCambio
1	1001	NULL	1	2025-05-20 09:15:21
2	1001	1	2	2025-05-20 10:30:45
3	1001	2	3	2025-05-20 14:22:10
4	1002	NULL	1	2025-05-21 08:10:05



El historial permite reconstruir la línea de tiempo completa de cada expediente, clave para auditorías internas y trazabilidad de procesos académicos.

2 MODIFICACIÓN EVOLUTIVA DEL ESQUEMA (ALTER TABLE)

Durante el ciclo de vida del software, es común requerir nuevos atributos. Añadimos una columna a la tabla `Solicitud` sin eliminar ni afectar los datos existentes.

```
-- Agregar un nuevo atributo temporal para auditoría de tiempo real
ALTER TABLE Tramite.Solicitud
ADD FechaUltimaActualizacion DATETIME2 NULL;
GO
```

SQL

¿QUÉ SUCEDE AL EJECUTAR ALTER TABLE?



Se agrega la nueva columna al final de la tabla.



Los datos existentes se conservan sin cambios.



Los registros anteriores tendrán valor NULL en el nuevo campo.



La aplicación podrá usar este campo para registrar la última actualización.

ESTRUCTURA ANTES Y DESPUÉS

ANTES (Tramite.Solicitud)

Columna	Tipo de Dato
IdSolicitud	INT (PK)
IdEstudiante	INT
IdTipoTramite	TINYINT
IdEstadoSolicitud	TINYINT
FechaRegistro	DATETIME2
NumeroExpediente	VARCHAR(20)
Observaciones	NVARCHAR(500)
MontoPago	DECIMAL(10,2)



DESPUÉS (Tramite.Solicitud)

Columna	Tipo de Dato
IdSolicitud	INT (PK)
IdEstudiante	INT
IdTipoTramite	TINYINT
IdEstadoSolicitud	TINYINT
FechaRegistro	DATETIME2
NumeroExpediente	VARCHAR(20)
Observaciones	NVARCHAR(500)
MontoPago	DECIMAL(10,2)
FechaUltimaActualizacion	DATETIME2 (NULL)

BENEFICIOS DE ALTER TABLE



Evolución sin Downtime:
Permite mejorar el esquema sin detener el sistema.



Datos Seguros:
No se pierden los datos existentes.



Trazabilidad Mejorada:
Campo útil para monitoreo y reportes en tiempo real.



BUENAS PRÁCTICAS DDL EN SQL AZURE



HAZ BACKUP ANTES DE CAMBIOS
Realiza copias de seguridad antes de ejecutar cambios DDL importantes.



PRUEBA EN ENTORNOS NO PRODUCTIVOS
Valida los cambios en desarrollo o staging antes de aplicarlos en producción.



USA MIGRACIONES CONTROLADAS
Gestiona los cambios de esquema con scripts versionados (ej. Flyway, EF Migrations).



REVISAS DEPENDENCIAS
Verifica objetos dependientes (vistas, procedimientos, reportes) después de modificar el esquema.



IMPLEMENTACIÓN DEL TRIGGER DE AUDITORÍA DE ESTADOS

Cada vez que el estado de una solicitud cambie, el trigger registrará automáticamente el cambio en la tabla de historial.

```
CREATE OR ALTER TRIGGER Tramite.TR_AuditarCambioEstado
ON Tramite.Solicitud
AFTER UPDATE
AS
BEGIN
    -- Evita el envío de mensajes de conteo de filas a la aplicación cliente
    SET NOCOUNT ON;

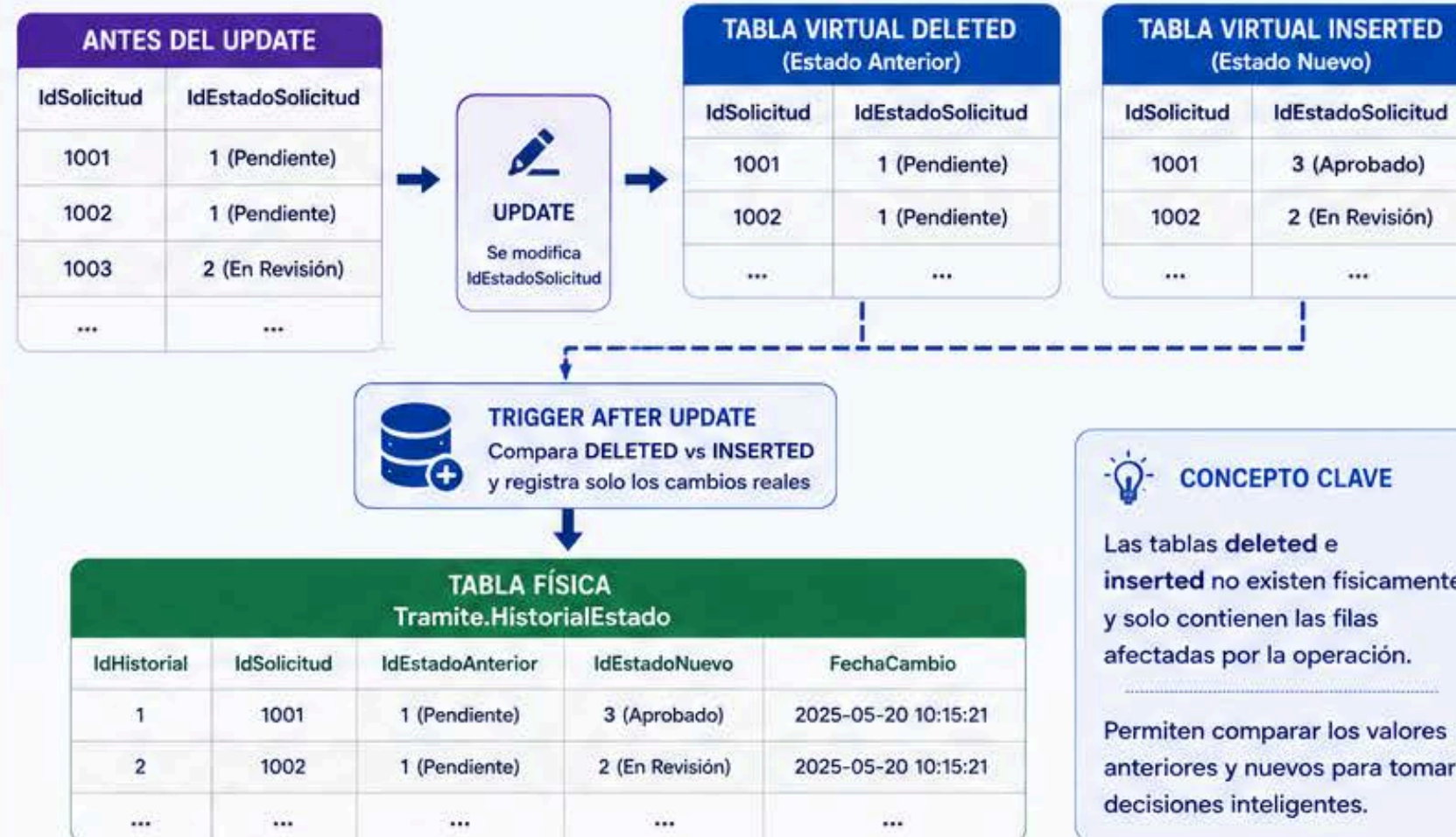
    -- Optimización: Solo ejecutar la lógica si la columna de estado fue modificada
    IF UPDATE(IdEstadoSolicitud)
    BEGIN
        -- Registra el cambio: estado anterior vs. estado nuevo
        INSERT INTO Tramite.HistorialEstado
        (
            IdSolicitud,
            IdEstadoAnterior,
            IdEstadoNuevo,
            FechaCambio
        )
        SELECT
            d.IdSolicitud,
            d.IdEstadoSolicitud,
            i.IdEstadoSolicitud,
            SYSDATETIME()
        FROM deleted d
        INNER JOIN inserted i
            ON i.IdSolicitud = d.IdSolicitud
        WHERE d.IdEstadoSolicitud <> i.IdEstadoSolicitud; -- Evita registros
                                                    -- redundantes si el valor
                                                    -- cambió

    END;
END;
GO
```

SQL

¿CÓMO FUNCIONA UN TRIGGER AFTER UPDATE?

Cuando se ejecuta una actualización (UPDATE) sobre la tabla `Tramite.Solicitud`, SQL Server (Azure SQL) crea dos tablas virtuales (en memoria) durante el proceso.



RESTRICCIÓN CLAVE EN SQL AZURE

Aunque los triggers DML estándar son completamente compatibles, ciertas opciones heredadas NO están admitidas en Azure SQL:



WITH APPEND
(No soportado)



NOT FOR REPLICATION
(No soportado)



Triggers globales ALL SERVER
(No soportado)



BUENAS PRÁCTICAS PARA TRIGGERS EN LA NUBE



1. MANTÉN LA LÓGICA LIGERA

Los triggers se ejecutan en la misma transacción; evita operaciones costosas y de larga duración.



2. USA CONDICIONALES

Utiliza `UPDATE(columna)` para ejecutar la lógica solo cuando sea estrictamente necesario.



3. ÍNDICES APROPIADOS

Asegúrate de tener índices en columnas utilizadas en joins y filtros dentro del trigger.



4. MONITOREO

Supervisa el rendimiento del trigger con Query Store y Extended Events para detectar problemas.



RESTRICCIONES CRÍTICAS DE DDL EN SQL AZURE

Al migrar código de base de datos a SQL Azure, la gran mayoría de los fallos de despliegue ocurren por intentar configurar aspectos de hardware o de almacenamiento físico.

OPERACIÓN DDL TRADICIONAL	ESTADO EN SQL AZURE	RAZÓN Y EXPLICACIÓN	ALTERNATIVA / SOLUCIÓN CLOUD
ON [PRIMARY] o ON [Filegroup]	RESTRINGIDO	No se pueden definir rutas de almacenamiento físico personalizadas. SQL Azure gestiona el almacenamiento automáticamente en la nube.	Almacenamiento administrado SQL Azure distribuye y replica los datos automáticamente para garantizar alta disponibilidad.
CREATE TRIGGER ... ON ALL SERVER	NO COMPATIBLE	Los triggers DDL con ámbito de servidor comprometerían la seguridad multi-inquilino (multi-tenant) de la infraestructura de Azure.	Triggers a nivel de base de datos Use triggers DDL en el ámbito de la base de datos actual solamente.
WITH ENCRYPTION	NO RECOMENDADO (Limitado)	Aunque es compatible con algunos objetos, no protege los datos en sí. No es la mejor práctica en la nube.	Always Encrypted + TDE Use Always Encrypted para columnas sensibles y TDE (Transparent Data Encryption) para datos en reposo.
DROP DATABASE	COMPATIBLE	Elimina la base de datos de manera lógica. El control de restauración, backups e históricos se realiza a través del portal de Azure o ARM Templates.	Azure Portal / ARM Templates Gestione eliminaciones, backups y retenciones usando herramientas nativas de Azure.

LECCIÓN DE DISEÑO ARQUITECTÓNICO

En SQL Azure, la separación por esquemas lógicos reemplaza la necesidad de segregar datos en múltiples bases de datos físicas o archivos de disco distintos.

ENFOQUE TRADICIONAL (ON-PREMISE)

Segregación física por bases de datos o archivos

BD_Tramite
 (Archivo / Disco A)

BD_Academico
 (Archivo / Disco B)

BD_Catalogo
 (Archivo / Disco C)

ENFOQUE EN AZURE (CLOUD)

Separación lógica por esquemas en una misma base de datos

Base de Datos: SistemaGradosTitulos

Esquema: Tramite

- Solicitud
- HistorialEstado
- ...

Esquema: Academico

- Estudiante
- Persona
- ...

Esquema: Catalogo

- EstadoSolicitud
- TipoTramite
- ...

BENEFICIOS DEL ENFOQUE CLOUD

Administración simplificada
 Menos objetos físicos que mantener y monitorear.

Seguridad y permisos centralizados
 Controle accesos por esquema, usuario o rol de forma granular.

Escalabilidad transparente
 Azure escala automáticamente almacenamiento y rendimiento.

Mantenibilidad y despliegue ágil
 Deploys consistentes con scripts y DevOps.



Interacción Controlada con la Base de Datos

En SQL Azure, cada consulta mal estructurada que realiza un escaneo completo de tabla (table scan) consume unidades de procesamiento de la base de datos (DTUs/vCores). Por ello, el uso de filtros precisos en la cláusula WHERE es fundamental.

1



Consulta Segmentada (SELECT)

Filtra los registros activos de manera que el motor pueda utilizar los índices non-clustered sin sobrecargar la memoria.

```
SELECT IdSolicitud, IdEstudiante, FechaRegistro, NumeroExpediente
FROM Tramite.Solicitud
WHERE IdEstadoSolicitud = 1; -- Consultar únicamente solicitudes en estado 'Iniciado'
```



- ✓ Usa índices
- ✓ Reduce table scans
- ✓ Menor consumo de memoria

2



Modificación Segura (UPDATE)

Actualiza únicamente la fila específica identificada por su clave primaria, registrando la fecha exacta del cambio.

```
UPDATE Tramite.Solicitud
SET IdEstadoSolicitud = 2,
    FechaUltimaActualizacion = SYSDATETIME()
WHERE IdSolicitud = 1; -- El filtro por ID previene la alteración accidental de otros expedientes
```



- ✓ Precisión
- ✓ Control
- ✓ Integridad

WHERE por clave primaria protege la integridad

3



Depuración Dirigida (DELETE)

Remueve físicamente registros huérfanos o requisitos inválidos utilizando siempre un identificador único.

```
DELETE FROM Documento.RequisitoSolicitud
WHERE IdRequisitoSolicitud = 10;
```



Eliminar solo con identificador único
Evita borrados masivos accidentales.

Patrón DML Seguro en SQL Azure



Filtro WHERE preciso

Restringe el conjunto de registros al mínimo necesario.



Uso eficiente de índices

El motor utiliza índices non-clustered para localizar los datos.



Menor CPU / memoria

Disminuye table scans y el consumo de DTUs/vCores.



Mayor integridad

Operaciones precisas que protegen la integridad física de las tuplas.

Buenas Prácticas DML en SQL Azure



Usar siempre cláusulas **WHERE** explícitas.



Validar previamente los registros con **SELECT** antes de **UPDATE** o **DELETE**.



Aprovechar claves primarias e índices non-clustered.



Evitar operaciones masivas sin respaldo o transacción controlada.



Registro Atómico de Solicitudes (TRY / CATCH)

Cuando un estudiante inicia un trámite, el sistema debe insertar la cabecera del expediente en Tramite.Solicitud y, simultáneamente, su estado inicial en Tramite.HistorialEstado. Si alguna de estas operaciones falla, la base de datos revierte los cambios pendientes para prevenir estados inconsistentes o registros huérfanos.



Principio ACID:
todo o nada

Implementación SQL (TCL con TRY / CATCH)

```
1 BEGIN TRY
2   -- 1. Iniciar el bloque transaccional
3   BEGIN TRANSACTION;
4
5   -- Registrar la solicitud principal
6   INSERT INTO Tramite.Solicitud
7   (
8     IdEstudiante,
9     IdTipoTramite,
10    IdEstadoSolicitud,
11    FechaRegistro,
12    NumeroExpediente
13  )
14  VALUES
15  (
16    1,
17    2,
18    1,
19    SYSDATETIME(),
20    'EXP-2026-0001'
21  );
22
23  -- Obtener el ID autogenerado en la sesión actual
24  DECLARE @IdSolicitud INT = SCOPE_IDENTITY();
25
26  IdSolicitud,
27  IdEstadoAnterior,
28  IdEstadoNuevo,
29  FechaCambio
30  )
31  VALUES
32  (
33    @IdSolicitud,
34    NULL,
35    1,
36    SYSDATETIME()
37  );
38
39  -- 2. Confirmar permanentemente todos los cambios
40  COMMIT TRANSACTION;
41
42  END TRY
43  BEGIN CATCH
44    -- 3. Si ocurre algún error en el bloque TRY, revertir los cambios pendientes
45    IF @@TRANCOUNT > 0
46      ROLLBACK TRANSACTION;
47
48    -- Propagar el error original a la aplicación cliente
49    THROW;
50  END CATCH;
```

Flujo Transaccional: todo o nada



Atomicidad

Las dos inserciones se comportan como una sola unidad lógica.



Consistencia

Se evita crear solicitudes sin historial inicial.



Recuperación ante errores

TRY / CATCH y ROLLBACK protegen la integridad del sistema.



Trazabilidad

Cada trámite nace con su primer hito en HistorialEstado.



Buenas prácticas en la nube



Usa transacciones solo durante el tiempo estrictamente necesario.



Combina TRY/CATCH con THROW para propagar errores reales.



Evita transacciones largas que incrementen bloqueos y consumo.



Valida siempre la integridad entre tablas relacionadas.



Seguridad lógica y granular: el usuario correcto accede solo a lo que necesita

EL PRINCIPIO DE MÍNIMO PRIVILEGIO

Reemplazamos el uso de cuentas con privilegios elevados por roles lógicos con permisos limitados y específicos por esquema.

USUARIOS DE LA APLICACIÓN

Secretaria de Grados

Operador de Trámite

Supervisor Académico

Solo Lectura (Consultas)

ROLES DE BASE DE DATOS

RolSecretariaGrados
(DML parcial + Lectura)

RolOperadorTramite
(DML completo en Trámite)

RolSupervisorAcademico
(Lectura académica + Reportes)

RolSoloConsulta
(Solo SELECT en esquemas)

ESQUEMAS DE LA BASE DE DATOS

Tramite

Solicitud

HistorialEstado

Documento

TipoTramite

...

Academico

Estudiante

Persona

Nota

Programa

...

Catalogo

EstadoSolicitud

TipoDocumento

Ubigeo

...

TIPO DE ACCESO

SELECT, INSERT, UPDATE

SELECT, INSERT, UPDATE

SELECT

SELECT

SELECT

-- SIN ACCESO --

SELECT

SELECT

SELECT

SELECT

SELECT

SELECT

Dividir los accesos por ESQUEMAS lógicos (Tramite, Academico, Catalogo) simplifica la administración, fortalece la seguridad y facilita las auditorías.

IMPLEMENTACIÓN CON T-SQL: ROLES Y PERMISOS

1

Crear el rol específico para el personal administrativo.

```
-- 1. Crear el rol específico para el personal administrativo
CREATE ROLE RolSecretariaGrados;
GO
```

2

Otorgar permisos de consulta, inserción y modificación sobre el módulo de trámites.

```
-- 2. Otorgar permisos de consulta, inserción y modificación
---- sobre el módulo de trámites
GRANT SELECT, INSERT, UPDATE
ON SCHEMA::Tramite
TO RolSecretariaGrados;
GO
```

3

Otorgar permisos de solo lectura sobre el módulo académico.

```
-- 3. Otorgar permisos de solo lectura
---- sobre el módulo académico
GRANT SELECT
ON SCHEMA::Academico
TO RolSecretariaGrados;
GO
```

4

Restringir explícitamente la eliminación física de cualquier registro del módulo.

```
-- 4. Restringir explícitamente la eliminación física
DENY DELETE
ON SCHEMA::Tramite
TO RolSecretariaGrados;
GO
```

BUENA PRÁCTICA: Evita dar permisos directos a usuarios. Agrúpalos en ROLES. Así, gestionar cambios es más rápido y seguro.

DML: Consultas eficientes y selectivas

- ✓ Utiliza SELECT especificando las columnas exactas requeridas.
- ✓ Evita SELECT * para optimizar el ancho de banda y la memoria RAM asignada en Azure.
- ✓ Usa filtros precisos (WHERE) para aprovechar los índices.

TCL: Transacciones para garantizar integridad

- ✓ Usa BEGIN TRY / CATCH en todas las operaciones que involucren múltiples tablas.
- ✓ Asegura el principio ACID: las operaciones son todas exitosas o ninguna lo es.
- ✓ Previene bloqueos prolongados y mantiene la consistencia.

DCL: Seguridad por esquemas y roles

- ✓ Agrupa permisos por ESQUEMAS (no por tablas).
- ✓ Aplica el principio de Mínimo Privilegio.
- ✓ Facilita la auditoría, el cumplimiento normativo y el mantenimiento de la seguridad.